

Table 5. Overview of QWEN3’s run results.

Run	Duration	Performed Rounds			Results			Tokens Planner		Tokens Executor		Cost	
		PLANNER	EXECUTOR	Commands	Done	Almost	Lead	Prompt	Compl.	Prompt	Compl.	Cost	per User
run-20250523-084832	9007.15	92	2.03 ± 0.35	1.04 ± 0.33	0	0	1	343.48	29.33	251.49	230.37	\$ 3.21	
run-20250523-112021	5380.98	29	2.00 ± 1.22	1.03 ± 1.18	0	0	1	93.41	91.13	93.43	53.75	\$ 1.81	
run-20250523-141744	649.59	9	1.78 ± 0.67	0.89 ± 0.33	0	0	0	39.44	4.71	24.73	12.49	\$ 0.23	
run-20250606-072612	7428.48	14	2.86 ± 1.03	1.86 ± 1.03	0	0	0	73.05	91.06	88.86	111.98	\$ 2.22	
run-20250606-093048	7157.45	79	2.95 ± 0.55	1.96 ± 0.49	0	0	1	289.32	19.75	392.14	204.53	\$ 2.51	
run-20250606-123053	7178.42	58	4.57 ± 0.96	3.59 ± 0.96	0	0	1	249.37	34.96	553.1	130.84	\$ 1.89	
Average	6133.68	46.83	2.84 ± 1.21	1.86 ± 1.19	0	0	0.66	181.34 ± 128.20	45.16 ± 37.03	233.96 ± 205.80	123.99 ± 84.99	\$ 1.98 ± \$ 1.00	

Executed Commands are summarized per PLANNER-Round. Within results, *done* designates fully compromised user accounts, *almost* attacks that failed due to a minimal error, and *leads* are designated as concrete vulnerabilities that the PLANNER has included within the PTT to follow-up to (detailed in Section 3.5. All Token costs are given in kilo-Tokens (kTokens).

theres and *leads*. GPT-4o also pursued diverse multi-modal attacks such as social-engineering and web penetration-testing (Section 6.3.1).

5.2 Reasoning SLM: QWEN3:32b

We used QWEN3 as an example of a locally-run small language model (SLM). It also makes for our second open-weight model (next to DEEPSEEK-V3). Results of QWEN3 were substantially worse than the results of the other models. It was the only model that produced not a single *compromised user account* nor *almost there*. To foreshadow the qualitative discussion in Section 6.1, QWEN3 possesses sufficient penetration-testing knowledge as indicated by the execution traces but was not able to successfully integrate the EXECUTOR’s results into the PTT, leading to the PLANNER issuing the same tasks for the EXECUTOR repeatedly.

Another problematic behavior was that QWEN3 sometimes ignored the scenario prompt and went off the rails, either switching the PLANNER’s goals (Section 6.2.5), ignoring safety instructions (Section 6.5), or hallucinating successful compromise of the target network.

Given the presented results, we do not include QWEN3 in the subsequent discussion of reasoning LLMs but discuss it in a separate Section 6.1.

5.3 Reasoning LLMs: OpenAI o1+GPT-4o and Google GEMINI-2.5-FLASH (preview)

Reasoning models include techniques such as Chain-of-Thought (CoT) or Reflexion (Section 2.3.2) to inherently include optimizations that were previously applied through prompt-engineering for traditional LLMs. For evaluation of reasoning LLMs, we used a combination of OpenAI’s o1 and GPT-4o, as well as Google’s GEMINI-2.5-FLASH in a preview version. The former is a dedicated reasoning model with very premium pricing. We followed best engineering practices and used o1 for strategic reasoning tasks (PLANNER) and combined it with a traditional non-reasoning LLM for the EXECUTOR. GEMINI-2.5-FLASH is a combined model suitable for both reasoning- and non-reasoning tasks.

5.3.1 Compared to Non-Reasoning Models. Our results show that, compared to non-reasoning models, reasoning models were able to compromise substantially more accounts as well as provide double the leads. They were able to perform substantially more high-level PLANNER rounds than non-reasoning LLMs (further discussed in Section 6.2). They consumed and produced substantially more tokens; especially the PLANNER’s output was significantly higher

Table 6. Overview of GEMINI-2.5-FLASH’s run results.

Run	Performed Rounds			Results			Tokens Planner		Tokens Executor		Cost	
	PLANNER	EXECUTOR	Commands	Done	Almost	Lead	Prompt	Compl.	Prompt	Compl.	Cost	per User
run-20250519-091544	77	4.79 ± 3.25	3.79 ± 3.25	1	1	8	2552.33	1176.44	847.66	37.09	\$ 2.96	\$ 2.96
run-20250519-140037	41	3.39 ± 2.45	2.39 ± 2.45	0	4	4	815.34	314.54	549.7	16.59	\$ 1.41	
run-20250520-080005	77	3.45 ± 2.51	2.47 ± 2.50	1	2	6	2126.15	971.17	623.73	35.10	\$ 3.21	\$ 3.21
run-20250520-104815	47	3.38 ± 2.35	2.38 ± 2.35	1	0	4	1082.06	481.61	373.17	21.98	\$ 1.60	\$ 1.60
run-20250520-131807	56	3.91 ± 2.88	2.91 ± 2.88	1	2	4	2230.84	1150.72	540.05	91.21	\$ 3.56	\$ 3.56
run-20250520-152006	77	3.60 ± 2.40	2.61 ± 2.39	1	4	7	2385.87	1046.11	886.15	50.04	\$ 3.48	\$ 3.48
Average	62.5	3.81	2.82	0.83	2.16	5.50	1865.43	856.77	636.74	42.0	\$ 2.7	\$ 2.96
		± 2.72	± 2.72				± 729.46	± 366.68	± 196.6	± 26.85	± \$ 0.95	

Executed Commands are summarized per PLANNER-Round. Within results, *done* designates fully compromised user accounts, *almost* attacks that failed due to a minimal error, and leads are designated as concrete vulnerabilities that the PLANNER has included within the PTT to follow-up to (detailed in Section 3.5. All Token costs are given in kilo-Tokens (kTokens).

compared to non-reasoning models, indicating both a more detailed PTT and increased context information for the EXECUTOR.

5.3.2 Comparing o1+GPT4o and GEMINI-2.5-FLASH. Both models yielded similar results but the combination of OpenAI’s o1+GPT-4o was able to compromise double the accounts compared to GEMINI-2.5-FLASH. Qualitative Analysis will show (Section 6.2) that GEMINI-2.5-FLASH’s PLANNER offered more stable trajectories and hyper-focused upon a single AD domain controller/domain while o1+GPT-4o was less stable and able to attack more low-hanging fruits by jumping between AD controllers/domains. GEMINI-2.5-FLASH’s EXECUTOR performed less rounds and executed less commands, indicating more targeted task and command selection. Overall, Gemini was able to execute 50% more high-level strategy rounds, due to less rounds per EXECUTOR invocation and the higher server-side token throughput when compared to OpenAI’s cloud platform.

GEMINI-2.5-FLASH used substantially more tokens compared to o1+GPT-4o, esp. the PLANNER module which used roughly four times the tokens compared to o1. The EXECUTOR token usage was more comparable. While GEMINI-2.5-FLASH used substantially more tokens, its overall cost was an order of magnitude lower than o1+GPT-4o’s cost, due to the different pricing regimes imposed by the respective LLM providers.

5.3.3 Attack Vector Coverage. Figure 9 overviews the used attack vectors and indicates that both LLMs have sufficient background knowledge of hacking techniques and tooling. Further discussion, esp. on the reasoning LLM’s better overall performance, can be found in Sections 6.3 and 6.2.

5.4 PLANNER rounds, EXECUTOR rounds, and Command Counts

Our prototype incorporates three control loops situated on distinct abstraction layers. On the highest-level, the PLANNER control loop is responsible for selecting new tasks (“strategy round”). The PLANNER stops execution if no further leads are available for follow-up.

The task generated by the PLANNER is transmitted to the EXECUTOR who employs an LLM prompt to propose zero or more system commands to solve the received task. The result of the executed system commands is presented back to the EXECUTOR to decide whether to terminate or issue new commands. If the LLM detects the task to be solved, it can stop the EXECUTOR loop and return the result to the PLANNER. We enforce an upper bound of 10 EXECUTOR rounds. The used prompts are included in the Appendix, Sections A.4 and A.5.

Table 7. Overview of O1/GPT-4o’s run results.

Run	Performed Rounds			Results			Tokens Planner		Tokens Executor		Cost	
	PLANNER	EXECUTOR	Commands	Done	Almost	Lead	Prompt	Compl.	Prompt	Compl.	Cost	per User
run-20250128-181630	36	4.50 ± 3.37	4.42 ± 4.25	3	2	6	373.02	207.58	417.12	57.8	\$ 18.30	\$ 6.10
run-20250128-203002	25	3.96 ± 2.75	4.20 ± 3.85	2	1	6	179.44	110.93	191.65	12.21	\$ 9.30	\$ 4.65
run-20250129-085237	61	5.62 ± 3.31	5.44 ± 3.22	1	3	10	808.05	426.38	774.25	39.32	\$ 35.68	\$ 35.68
run-20250129-110006	66	4.02 ± 2.46	3.71 ± 2.66	1	1	7	653.22	408.43	687.06	33.64	\$ 33.39	\$ 33.39
run-20250129-152651	48	5.46 ± 3.33	5.40 ± 3.59	3	2	6	584.99	303.96	692.16	57.60	\$ 26.07	\$ 8.69
run-20250129-194248	38	3.87 ± 2.44	3.92 ± 2.76	1	2	5	338.78	200.34	315.74	33.04	\$ 16.9	\$ 16.9
Average	45.67	4.66 ± 3.04	4.56 ± 3.37	1.83	1.83	6.66	489.58	276.27	513.0	38.94	\$ 23.28	\$ 17.56
							± 232.3	± 125.37	± 237.49	± 17.22	± \$ 10.24	

Executed Commands are summarized per PLANNER-Round. Within results, *done* designates fully compromised user accounts, *almost* attacks that failed due to a minimal error, and leads are designated as concrete vulnerabilities that the PLANNER has included within the PTT to follow-up to (detailed in Section 3.5. All Token costs are given in kilo-Tokens (kTokens).

The number of EXECUTOR rounds and the number of executed commands can differ. This can happen if the EXECUTOR issues multiple system commands within a round, or if the EXECUTOR issues no command, e.g., when it adds additional information to its history or produces a summary for the PLANNER. We do not cap the maximum number of executed system commands per strategy nor EXECUTOR round.

Log data shows that during a single strategy round, the EXECUTOR round is performed 3.93 times on average, i.e., that the EXECUTOR is able to finish a task within four rounds. This indicates that the EXECUTOR round limit can be raised from 10 as the additional rounds will be used by the EXECUTOR tries to repair invalid commands (Section 6.4.3). In the case of OpenAI’s o1+GPT-4o, increasing the number of EXECUTOR rounds thus should reduce the overall costs as this potentially decreases the number of expensive strategy rounds dealing with invalid commands. The similarity between the average number of measured EXECUTOR rounds and system calls indicate that parallel command execution is not a common occurrence.

On average, after two hours of execution the PTT contained sufficient leads (3.25 for non-reasoning LLMs, 6.08 for reasoning LLMs) to warrant longer execution times.

5.5 LLM Cost and Call Duration

Running our most expensive configuration (o1+GPT-4o) incurred an average cost of \$11.64 per hour while all other configurations were at least one order of magnitude cheaper. Even using our most expensive configuration’s cost compares favorable to those of professional penetration-testers (Section 2.2.2). Consequently, our focus was evaluating the overall feasibility of the prototype rather than cost optimization. Nonetheless, we conducted an initial analysis of both the monetary expenditures and the associated timing costs to ensure a comprehensive assessment of the system’s operational efficiency.

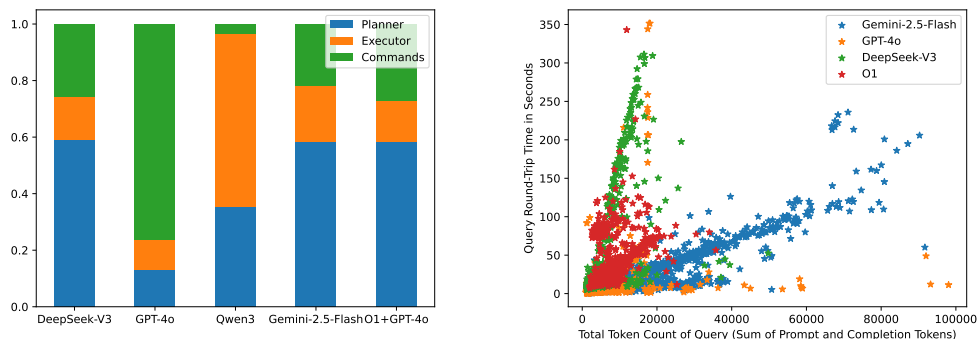
It is important to note that newer iterations of LLMs generally offer reduced operational costs and improved processing speeds, which may render immediate performance optimizations less critical.

5.5.1 LLM Costs. When analyzing the costs occurring during our two hour sampling runs, three distinct price points appear. The cheapest LLM to operate was DEEPSEEK-V3 with operating costs of around \$0.10 per hour. The second price point was around \$2.42 achieved by GPT-4o, GEMINI-2.5-FLASH, and QWEN3. The most expensive configuration was o1+GPT-4o operating at \$11.64 per hour. Using our most expensive configuration, the average cost for a fully compromised domain account during our evaluation was \$17.56. These compare favorably to the cost of employing

human penetration testers. This indicates that LLM-guided penetration testing tools can be employed to reduce time needed by professional penetration testers and can decrease the cost of security tests, potentially democratizing access to penetration testing, esp. for companies traditionally not able to afford these activities, e.g., NPOs and SMEs.

Further analyzing our most expensive model (o1+GPT-4o), 94.07% of the cost occurred through using its premium o1 reasoning model. Within the prototype, all o1 LLM prompting occurs within the PLANNER component. Furthermore, o1 bills output token cannot be prefix-cached.

5.5.2 Overall Time Consumption. Figure 10(a) highlights how time was spent by the different prototype configurations. We differentiate between time spent by the PLANNER, the EXECUTOR, and time spent waiting on command completion. DEEPSEEK-V3, GEMINI-2.5-FLASH, and the combined o1+GPT-4o prototype exhibit a similar behavior of spending 60% of their time on high-level strategy making (PLANNER), 15–20% of the time on selecting and analyzing commands (EXECUTOR), and finally waiting for commands to be finished for 20–25% of the time. QWEN3’s PLANNER is not incorporating the EXECUTOR’s information correctly, thus the PLANNER’s cost are lower and more time is spent executing commands. GPT-4o is a non-reasoning model and thus spends less time updating the PTT and selecting new tasks to be forwarded to the EXECUTOR.



(a) Time (in percent) spent in the different prototype areas. *Planner* includes all high-level decision making and strategizing, *Executor* includes all activity related to creating and analyzing command line tool worse with increased token counts compared to the over evaluated invocations, and *Commands* describes the “wait” time for executed LLMs. (b) LLM query roundtrip time in dependence of the utilized *total token* count (as reported by the respective LLM API). DEEPSEEK-V3 scales worse time-wise compared to the other models while GPT-4o is able to reach results in less time, matching Figure 10(a) showing less time spent by GPT-4o on LLM invocations overall. o1 is only used for high-level PTT tasks thus only operates on smaller input sizes for which it performs worse than GPT-4o and GEMINI-2.5-FLASH, presumably as it spends more time reasoning and updating attack strategies within the PTT.

Fig. 10. Our sampling runs were time-capped at two hours, making the efficacy of time spent by the LLMs of high importance. The graphs show in which areas the different LLMs spent their time in, and how long singular LLM queries took.

To further analyze the time consumption of our prototype we display the different roundtrip times imposed by the different models in dependence of the reported total token count (Figure 10(b)). Results indicate that DEEPSEEK-V3 scales worse time-wise compared to the other models while GPT-4o is able to reach results in less time, matching Figure 10(a) showing less time spent by GPT-4o on LLM invocations overall. o1 is only used for high-level PTT tasks thus only operates on smaller input sizes for which it performs worse than GPT-4o and GEMINI-2.5-FLASH, presumably as it spends more time reasoning and updating attack strategies within the PTT.

5.5.3 PTT Growth. Reducing the size of the used PTT is an obvious candidate for optimization: the PTT is used as input for both *update-plan* and *select-next-task* queries, while being the output of the initial *update-plan* query. Figure 11

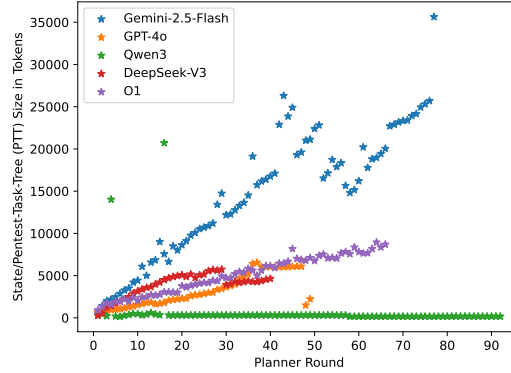


Fig. 11. The Pentest-Task-Tree (PTT) includes all current knowledge of the prototype about the target environment. Its size impacts both LLM runtime and costs. The graph shows the average size of the PTT per evaluated model in dependence of the currently executed PLANNER strategy-round.

shows the growth of the PTT per LLM during our sampling runs. GPT-4o, DEEPSEEK-V3, and o1 produced similar trajectories. QWEN3 was not able to integrate the EXECUTOR’s results into the PTT thus its PTT size never increased. The outlier was created through QWEN3 creating a PTT with repeated instructions. GEMINI-2.5-FLASH created longer and more convoluted PTTs compared to the other models.

5.5.4 Executor Context Size. The EXECUTOR prompt context increases with each of its performed rounds (capped at 10 rounds) as it incorporates both the executed commands as well as their output. Antagonistically, modern LLMs often perform prefix-caching that substantially reduces the costs for reoccurring prefixes, incentivizing append-only prompts. For example, OpenAI offers a 50% cost reduction on cached input tokens when using GPT-4o, while Google and DeepSeek offer up to 75% cost reductions.

Figure 12(a) shows the average prompt input size during run EXECUTOR rounds. Models performed similar with the exception of DEEPSEEK-V3 which used more tokens for later EXECUTOR rounds. Figure 12(b) shows the percentage of input tokens that were cached by the different models. QWEN3 using Ollama did not report prefix caching. DEEPSEEK-V3 and GPT-4o reported similar rates of around 80% input tokens automatically cached by prefix-caching while GEMINI-2.5-FLASH reported prefix caching rates of 10–15%.

5.6 Detailed Tool-Analysis for OpenAI o1+GPT-4o

Using professional penetration-testers, we performed a detailed analysis of command line tools employed by our evaluated LLMs. Due to the combination of this analysis being very time-intensive and the limited availability of penetration-testers being able to perform the analysis, we limited it to our best-performing LLM configuration of OpenAI’s o1+GPT-4o.

5.6.1 Tool Usage. Our analysis shows that 72 different command line tools have been used by the EXECUTOR to solve their given tasks. Table 8 shows the 15 most often executed commands. Figure 13 shows the relative inclusion of tools within our experiment runs: 42% of tools were included in two runs or more.